solvpath

CANDIDATE BRIEF

Senior Front-End Engineer — Take-home

solvpath · post-purchase platform

Thanks for making it this far. This exercise is meant to look like a slice of real work at solvpath, not a puzzle with one right answer. We're far more interested in how you think, what you choose to build, and the quality of the details than in whether you check every box.

Time: budget a weekend — roughly **8–12 focused hours**. Please don't grind past that; if you run low on time, prioritise and tell us what you'd do next.

About solvpath

solvpath is a post-purchase platform for e-commerce brands: once a shopper has bought something, we power everything that comes after — order tracking, returns and exchanges, subscriptions, and self-service resolution. The work is customer-facing, high-traffic, and full of real-world edge cases (flaky networks, partial data, money math, multi-step flows). This exercise lives right in that world.

What you'll build

A small customer-facing app with **two connected experiences**:

1. Orders dashboard

A list of the shopper's past orders. At minimum it should let someone:

- Browse their orders, with the key details for each (order number, date, items, total, status).
- Filter by status and search by order number or product.
- Move through more than one page of results.

2. Return / exchange flow

Starting from a delivered order, a guided flow to return or exchange items. Along the way the shopper:

- Picks which items (and how many of each) to send back.
- Tells us why.

- Chooses a resolution — **refund to original payment, exchange, or store credit**. Store credit comes with a **+10% bonus** on the returned value.
- Reviews everything and submits.

How you break these into screens, steps, or routes is up to you.

Design

There is no Figma or mockup — the UI is yours to design. We want to see your product sense and craft, not your ability to trace a comp. Aim for something clean, considered, and pleasant to use; treat it as if it were shipping to real customers.

If you'd like grounding in the domain, real post-purchase experiences worth a look include Shopify's Shop app, Amazon's returns flow, and returns products like Loop or Narvar. Use them as inspiration for how the domain behaves — please don't copy any specific brand's look.

We've included the solvpath logo (`solvpath-logo.png`) in this package — feel free to use it in your build.

Brand & colours

Please build to the solvpath palette below so every submission reads as the same product — we want to compare your execution, not your colour choices. A ready-to-use `brand-tokens.css` (CSS custom properties) is included in this package: drop it in and use the variables, or map them into your framework's theme (Tailwind, CSS-in-JS, SCSS — whatever you're using).

- **Primary — deep navy** `#0E2C3E` (hover `#17394E`): primary buttons and key chrome. White text on navy.
- **Accent — teal** `#28B0C8` : active states, focus rings, selection, small highlights. For teal text or links, use the deeper `#0C6E80` so it stays legible; soft tint `#E4F5F8` for subtle backgrounds.
- **Neutrals** — text `#0B2430` · muted `#566671` · borders `#E2E8EB` · page background `#F4F7F9` .
- **Status** — delivered `#127A3B` , in transit `#1D4ED8` , processing `#B45309` , cancelled `#B91C1C` (each pairs with a soft tint background).
- **Type** — Inter (or a close system sans).

Use navy for primary actions and teal as the accent; keep contrast accessible (the palette above is chosen so white-on-navy and teal-deep-on-white both pass).

The "backend"

You're given a mock API in `mockApi.ts` (see the starter kit README). It exposes:

- `listOrders({ page, pageSize, status, query, signal })` → a page of orders
- `getOrder(orderId, signal)` → a single order

- `submitReturn(request)` → a receipt

It behaves like a real API: requests take time, and some of them fail. Build against it as you would a real service.

Requirements

- **Both experiences work end to end** against the mock API.
- **The states that matter are handled** — not just the happy path. Loading, empty, and failure are part of the product, not afterthoughts.
- **The return flow validates as it goes** and computes the resolution amount correctly (mind the store-credit bonus, and remember money is in cents).
- **It's responsive** and usable on a phone as well as a laptop.
- **TypeScript throughout.**

Beyond that, the spec is **intentionally not exhaustive**. Where it's silent, make a reasonable decision, build it the way you think it should work, and jot down a line about why. We pay as much attention to the calls you make on your own as to the requirements above — a good engineer notices the things nobody told them to handle.

Tech

Your choice of stack. We work primarily in **React and Angular** — use whichever you're strongest in. TypeScript is required; styling approach, build tooling, and libraries are up to you. Reach for what lets you do your best work in the time.

What to hand in

1. **A Git repo** (GitHub/GitLab) with clear run instructions in the README (`install` → `run`).
2. **A short decision log** — a section in your README is perfect. What you chose to build, what you deliberately skipped, trade-offs you made, and what you'd do with another day.

How we'll look at it

We read the code, run the app, and read your decision log. We're looking for sound architecture, thoughtful state and data handling, correctness in the fiddly bits, real attention to UX, and clear reasoning. Polish in a few places you clearly cared about beats a thin, even coat everywhere.

You don't need authentication, a real backend, a test for every line, or feature-completeness beyond the above. Depth and judgment over breadth.

Have fun with it — and good luck.